Ex: 7
Date: 6/8/26

flow control and data link layer using sliding window.

Aim: To write a program to implement flow control at Data link layer using sliding window protocol and to simulate the flow of frames from one node to another

Algorithm:-

1) start the program
2) Read the total number of frames to be sent (N), the sender window size (w). and a frame number to simulate as lost during transmission.
3) Initialize base = 0 (sequence no. of the oldest un-acknowledged frame) and next seq = 0 (sequence number of the next frame to be sent.
4) Repeat while base < N:
   a) Determine the current sender window as the range [base, base + w - 1]
   b) send all frames with sequence numbers from next seq, increament next-seq after each time/frame, since the window

output :-

=== Sliding window Protocol — How control stimulation (Go-Back-N) ===

Enter total number of frams to send : 8

Enter sender's window size : 2

Enter frame number to simulate as Lost
(0 to 7, enter -1 for no loss) : 5

sender window : [frame 0 - frame 1]

Node A --> sending frame 0

Node A --> sending frame 1

Node B -> Received frame 1, sending ACK 1

Node A -> ACK 1 received. sliding window forward

sender window : [frame 2 - frame 3]

Node A --> sending frame 2

Node A --> sending frame 3

Node B -> Received frame 2, sending ACK 2

Node B -> Received frame 3, sending ACK 3

Node A -> ACK 3 received. sending window forward.

restricts the sender too at most - w outstanding frames at any time.

5) Repeat step 4 until all N frames have been transmitted.

6) stop the program.

Program :-

```
def simulate ( total_frames, window_size, lost_frame):

    base = 0
    next_seq = 0
    while base < total_frames :
        window_end = min ( base + window_size - 1,
                           total_frames - 1)
        print ("\n ----------")
        print (f" Sender window : [ frame {base} - frame {window_end}]")

        while next_seq < base + window_size and
                next_seq < total_frames:
            print ( f " Node A -> sending frame {next_seq}")
            if next_seq == lost_frame :
                print ( f " ( frame {next_seq} Lost in transit !) ")

            next_seq += 1
        if lost_frame is not None and base <=
                lost_frame < next_seq :
```

Sender window : [frame 4 - frame 5]

Node A -> sending frame 4

Node A -> sending frame 5
(frame 5 lost in transit !)

Node B -> Frame 5 missing . discarding out-of-order frame after it

Node A -> Time out ! . Resending window starting from frame 5 .

sender window : [frame 4 - frame 5]

Node A -> sending frame 5

Node B -> Received frame 4, sending ACK4

Node B -> Received frame 5, sending ACK5

Node A -> ACK 5 received . sliding window

sender window : [frame 6 - frame 7]

Node A -> sending frame 6

Node A -> sending frame 7

Node B -> Received frame 6, sending ACK 6

Node B -> Received frame 7, sending ACK 7

Node A -> ACK 7 received - sliding window forward .

All 8 frames Successfully transmitted from Node A to Node B .

```
        print(f"Node B -> Frame {lost_frame} missing. Discarding out-of-order frames after it.")

        print(f"Node A --> Timeout! Resending window starting from Frame {lost_frame}.")

        next_seq = lost_frame
        lost_frame = None
        continue

    for f in range(base, next_seq):
        print(f"Node B -> Received Frame {f}, sending ACK {f}")
        print(f"Node A --> ACK {next_seq - 1} received. Sliding window forward")
        base = next_seq

    print("\n----------")
    print(f"All {total_frames} frames successfully transmitted from Node A to Node B.")

def main():
    print("=== Sliding window protocol - Flow control Simulation (Go-back-N) ===\n")
```

```
print("\n ----------")
print(f"All {total_frames} frames successfully transmitted from Node A to Node B.")

def main():
    print("=== Sliding window Protocol - flow control simulation (Go-back-N) ===\n")
    total_frames = int(input("Enter total number of frames to send: "))
    window_size = int(input("Enter sender's window size: "))
    lost_frame = int(input(f"Enter frame number to simulate as lost \n (0 to {total_frames-1}, enter -1 for no loss): "))
    if lost_frame == -1:
        lost_frame = None
    simulate(total_frames, window_size, lost_frame)
if __name__ == "__main__":
    main()
```

Result :- The program to implement flow control to the data link layer using the sliding window protocol was successfully written in python, executed and verified by simulating the flow of frames between two nodes (Node A as sender and node B as receiver).